

StudentMaster Pro

Java project

Vityarthi

VIT-BHOPAL

18 SEP 2026

StudentMaster Pro

1. Cover Page:-

Project Title: StudentMaster Pro

Course: JAVA

Student Name: Himanshu

Registration Number: 25BAI11400

Faculty name: M.s Muthusundari

2. Introduction:-

StudentMaster Pro is a Java-based application designed to apply core Object-Oriented Programming concepts to educational administration. It provides a modular, reliable framework for processing student data.

3. Problem Statement:-

Manual grading and student record management are prone to human error and inefficiency. There is a need for a local, terminal-based system to standardize GPA calculations and securely manage student records in memory.

4. Functional Requirements:-

Module 1 (User Management): Administrators can input and store new student profiles.

Module 2 (Data Processing): The system dynamically converts raw cumulative marks into standardized percentages, letter grades, and 4.0 scale GPAs.

Module 3 (Reporting): The system generates a formatted, system-wide academic output.

5. Non-functional Requirements:-

Reliability: The system will not crash upon unexpected string inputs during integer collection.

Maintainability: Code is divided into 5 distinct classes separating models, logic, validation, and UI.

Usability: The CLI provides clear, conversational prompts and immediate feedback for invalid actions.

Performance: In-memory `ArrayList` structures ensure $O(1)$ insertion time for new student records.

6. System Architecture:-

The application utilizes a Monolithic CLI architecture. The `Main.java` class handles standard I/O, delegating data storage to `StudentManager.java` and complex math logic to `GradeCalculator.java`.

7. Design Diagrams:-

(Note to student: You must draw and insert simple images for these)*

Case Diagram: Admin -> [Add Student], [View Report].

Workflow Diagram: Start -> Menu -> Input Data -> Validate -> Process Grade -> Display -> End.

Class Diagram: `Main` (1) uses (1) `StudentManager` (1) has (Many) `Student`.

8. Design Decisions & Rationale:-

A modular OOP approach was chosen over a single procedural file. By separating `Input-Validator` and `GradeCalculator` from the main thread, the code becomes highly testable and allows for future expansion (e.g., adding GUI) without altering the core business logic.

9. Implementation Details:-

The project consists of 5 Java classes utilizing standard JDK libraries. `java.util.Scanner` handles parsing, while custom loops in `InputValidator` ensure the program state remains stable during user error.

10. Results:-

Satisfactory uptime more than 95 percent.

11. Testing Approach:-

Validation testing was conducted extensively on the `InputValidator` class to ensure negative bounds (e.g., entering -5 for a subject mark) are caught and handled gracefully without throwing `InputMismatchException`.

12. Challenges Faced:-

Ensuring the standard input buffer (newline characters) was properly consumed between `nextInt()` and `nextLine()` calls within the `Scanner` implementation required careful debugging.

13. Learnings & Key Takeaways:-

Successfully applied single-responsibility principles in Java, demonstrating that separating validation, processing, and data models leads to significantly cleaner and more scalable code.

14. Future Enhancements:-

Future iterations could implement a database (e.g., MySQL via JDBC) to persist records between sessions, or a graphical interface using JavaFX.

1. References:-

Java Documentation: Collections Framework

"BuildYourOwnProjectVITyarthi.pdf" Guidelines

References